UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

ACTIVIDAD 2. MODELADO Y CÓDIGO DE PROBLEMAS DE BÚSQUEDA

Dos problemas modelados como grafos y solución en Python

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Mtro. Javier García Lara

Programa: Ingeniería en Tecnologías de la Información e Innovación Digital

Grupo: 171 **Cuatrimestre:** 7

Nombre del alumno: Suarez Melonie Magdalena

Fecha: 19 de octubre de 2026

1. Introducción

Los algoritmos de búsqueda permiten explorar alternativas para encontrar un estado objetivo. Para resolver un problema, primero se define el estado inicial, las acciones posibles, las restricciones y la meta. Después se representa el problema mediante un grafo: los nodos son estados y las aristas son conexiones o acciones. En este trabajo se modelan dos problemas sencillos y se comparan los resultados de búsqueda en anchura (BFS) y búsqueda en profundidad (DFS).

2. Objetivo

Representar dos problemas como grafos, implementar algoritmos BFS y DFS en Python, comprobar sus recorridos y analizar la complejidad temporal y espacial.

3. Problema 1: encontrar una ruta entre lugares

Enunciado. Una persona necesita trasladarse desde el punto A hasta el punto G usando una red de caminos. Se busca encontrar una ruta válida y conocer cuál tiene menos aristas cuando todos los caminos se consideran del mismo costo.

Estados: lugares A, B, C, D, E, F y G. **Estado inicial:** A. **Meta:** G. **Acciones:** desplazarse a un lugar conectado. **Costo:** 1 por cada camino recorrido.

Grafo dirigido/no dirigido: se considera no dirigido, porque cada camino puede recorrerse en ambos sentidos. Conexiones: A-B, A-C, B-D, B-E, C-F, D-G, E-G y F-G.

4. Código Python BFS y DFS

El código usa una lista de adyacencia y reconstruye la ruta encontrada. BFS explora por niveles y devuelve la ruta con menos aristas en un grafo no ponderado. DFS explora una rama antes de retroceder; encuentra una ruta, pero no garantiza que sea la más corta.

```python
from collections import deque

grafo = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B", "G"],
    "E": ["B", "G"],
    "F": ["C", "G"],
    "G": ["D", "E", "F"]
}

def reconstruir_ruta(padres, meta):
    ruta = []
    actual = meta
    while actual is not None:
        ruta.append(actual)
        actual = padres[actual]
    return list(reversed(ruta))

def bfs(grafo, inicio, meta):
    cola = deque([inicio])
    padres = {inicio: None}
    while cola:
        actual = cola.popleft()
        if actual == meta:
            return reconstruir_ruta(padres, meta)
        for vecino in grafo[actual]:
            if vecino not in padres:
                padres[vecino] = actual
                cola.append(vecino)
    return None

def dfs(grafo, inicio, meta):
    pila = [inicio]
    padres = {inicio: None}
    while pila:
        actual = pila.pop()
        if actual == meta:
            return reconstruir_ruta(padres, meta)
        for vecino in reversed(grafo[actual]):
            if vecino not in padres:
                padres[vecino] = actual
                pila.append(vecino)
    return None

print("Ruta encontrada con BFS:", bfs(grafo, "A", "G"))
print("Ruta encontrada con DFS:", dfs(grafo, "A", "G"))
```

5. Resultados y análisis del problema 1

Con el orden de vecinos definido en el código, BFS encuentra una ruta de dos aristas, por ejemplo A-B-D-G. La ruta más corta tiene tres movimientos? No: la ruta A-B-D-G contiene tres aristas; otras rutas como A-B-E-G o A-C-F-G también contienen tres. Por eso la distancia mínima de A a G es de 3 aristas. DFS también encuentra una ruta válida, aunque el recorrido depende del orden de los vecinos. En un grafo con V vértices y E aristas, BFS y DFS tienen complejidad O(V + E) si se utiliza una lista de adyacencia.

6. Problema 2: encontrar una secuencia de acciones en un laberinto

Enunciado. Un robot se encuentra en una cuadrícula y debe llegar desde S (inicio) hasta M (meta), evitando las celdas bloqueadas (#). Puede moverse arriba, abajo, izquierda o derecha; no puede salir de la cuadrícula ni atravesar obstáculos.

Representación: cada celda libre es un estado; las conexiones entre celdas vecinas son acciones. Estado inicial: (0, 0). Meta: (2, 3). El costo de cada movimiento es 1. Se emplea BFS para encontrar el camino con menos movimientos.

Laberinto de ejemplo:

S . . #
. .
. . . M

En la cuadrícula anterior, las filas y columnas se numeran desde cero. La celda (0,0) es S; la celda (2,3) es M; el símbolo # representa una pared.

7. Código Python — búsqueda BFS en cuadrícula

```python
from collections import deque

laberinto = [
    ["S", ".", ".", "#"],
    ["#", ".", ".", "#"],
    [".", ".", ".", "M"]
]

inicio = (0, 0)
meta = (2, 3)

def bfs_laberinto(tablero, inicio, meta):
    cola = deque([inicio])
    padres = {inicio: None}
    movimientos = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    while cola:
        fila, columna = cola.popleft()
        actual = (fila, columna)

        if actual == meta:
            ruta = []
            while actual is not None:
                ruta.append(actual)
                actual = padres[actual]
            return list(reversed(ruta))

        for df, dc in movimientos:
            nf, nc = fila + df, columna + dc
            vecino = (nf, nc)
            dentro = 0 <= nf < len(tablero) and 0 <= nc < len(tablero[0])

            if dentro and tablero[nf][nc] != "#":
                if vecino not in padres:
                    padres[vecino] = (fila, columna)
                    cola.append(vecino)
    return None

ruta = bfs_laberinto(laberinto, inicio, meta)

if ruta:
    print("Ruta encontrada:", ruta)
    print("Cantidad de movimientos:", len(ruta) - 1)
else:
    print("No existe una ruta.")
```

8. Resultados y análisis del problema 2

El algoritmo evita las celdas marcadas con # y explora primero las posiciones más cercanas al inicio. En este laberinto existe una ruta hasta M: el robot puede avanzar por la parte central y después dirigirse hacia la meta. BFS devuelve la ruta con el menor número de movimientos porque todos los movimientos tienen costo unitario. Si se agregaran costos diferentes —por ejemplo, algunas celdas fueran más difíciles de cruzar— sería conveniente usar Dijkstra o A* con una heurística adecuada.

9. Comparación de algoritmos

Característica	BFS (anchura)	DFS (profundidad)
Estructura	Cola FIFO	Pila LIFO
Estrategia	Explora por niveles	Profundiza por una rama
Ruta más corta	Sí, si todas las aristas cuestan lo mismo	No la garantiza
Complejidad con lista de adyacencia	O(V + E)	O(V + E)
Uso de memoria	Puede guardar muchos nodos de la frontera	Puede usar menos memoria según el grafo y la implementación

10. Conclusiones

El modelado mediante grafos permite representar de forma clara los estados y las acciones disponibles en un problema. BFS resulta útil cuando se necesita una ruta mínima en problemas donde todos los movimientos tienen el mismo costo. DFS es una alternativa sencilla para explorar conexiones y encontrar alguna solución, aunque no asegura la ruta más corta. En el laberinto, BFS considera las restricciones de movimiento y evita los obstáculos. Antes de aplicar estos algoritmos a casos reales, es importante validar la representación del problema y probar situaciones donde exista o no exista una solución.

11. Declaración de uso de inteligencia artificial

Se utilizó ChatGPT como apoyo para estructurar el reporte, revisar la explicación de los algoritmos y preparar una versión inicial del código.

12. Referencias (formato APA 7)

Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Python Software Foundation. (s. f.). *collections — Container datatypes*. Python documentation. https://docs.python.org/3/library/collections.html

Python Software Foundation. (s. f.). *Data structures*. Python tutorial. https://docs.python.org/3/tutorial/datastructures.html